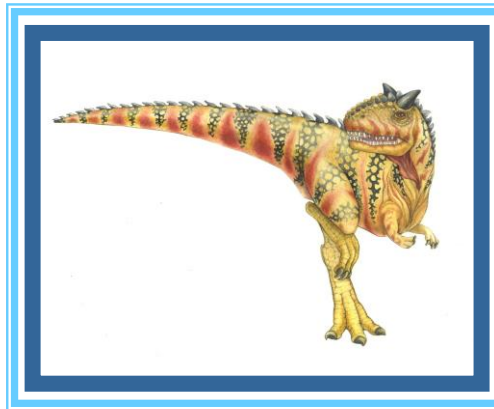


9. Bölüm: Sanal Bellek

Dr. Öğr. Üyesi Ertan Bütün





Sanal Bellek

- Background
- Demand Paging
- Copy-on-Write
- Page Replacement
- Allocation of Frames
- Thrashing





Background

- Komutların yürütülmesi için fiziksel bellekte olması gerekir; ancak bir programın boyutu fiziksel bellek boyutuyla sınırlı olduğundan bellekte yeteri kadar yer olmaması bir sorundur.
- Aslında gerçek programlar incelendiğinde çoğu durumda programın tümünün gerekli olmadığını görülmüştür. Örneğin:
 - Programların olağandışı hata durumları pratikte nadiren meydana geldiğinden, bu hataları ele alan program kod kısımları neredeyse hiç çalıştırılmaz.
 - Diziler, listeler ve tablolara genellikle ihtiyaç duyduklarından daha fazla bellek ayrılır.
 - Bir programın belirli seçenekleri ve özellikleri nadiren kullanılabilir. Örneğin, ABD devlet bilgisayarlarında bütçeyi dengeleyen rutinler uzun yıllardır kullanılmamaktadır.





Background

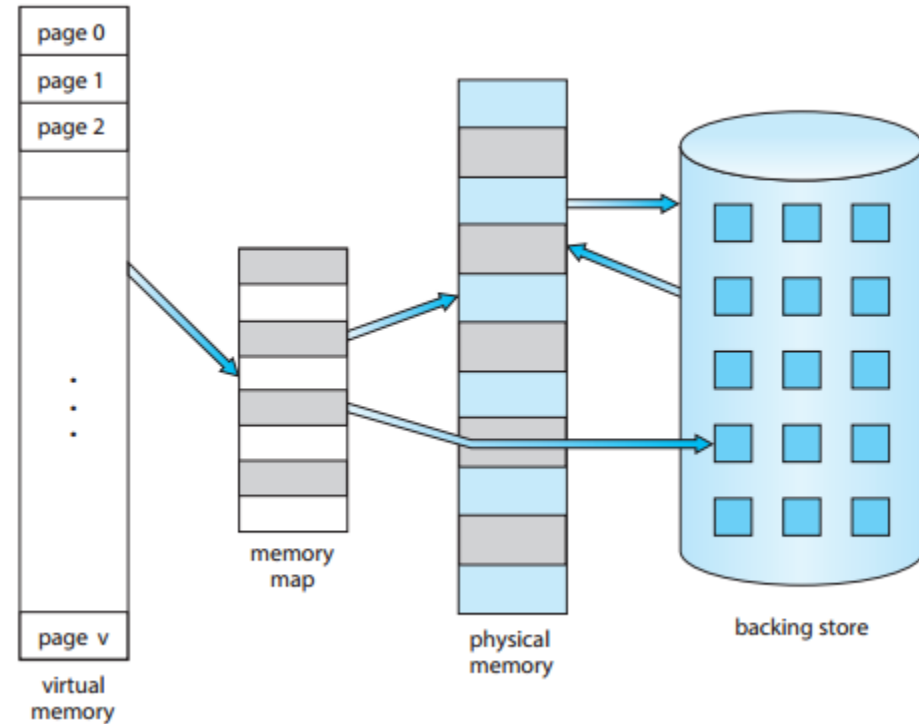
- Tüm programın gerekli olduğu durumlarda bile, **hepsine aynı anda** ihtiyaç duyulmayabilir.
- Yalnızca kısmen bellekte olan bir programı yürütebilmek birçok fayda sağlar:
 - Bir program artık mevcut fiziksel bellek miktarıyla kısıtlanmaz.
 - Her program çalışırken daha az bellek kullanır -> aynı anda daha fazla program çalışır.
 - ▶ Yanıt süresinde veya geri dönüş süresinde herhangi bir artış olmaksızın CPU kullanımı ve throughput artar.
 - Programların belli kısımlarını belleğe yüklemek veya swap etmek daha az I/O gerekir, böylece her program daha hızlı çalışır.





Virtual memory

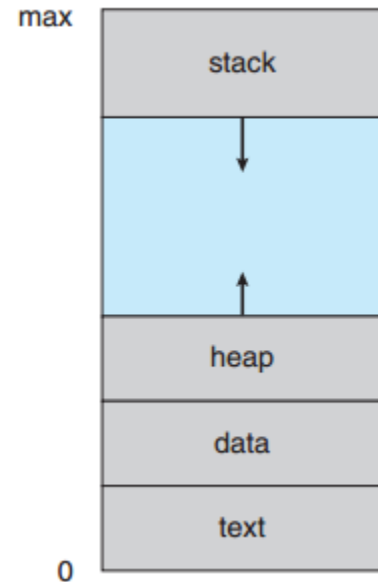
- **Sanal Bellek (Virtual memory)** – geliştiriciler tarafından algılanan mantıksal belleğin fiziksel bellekten ayrılmasıdır.
- Bu ayırım, küçük bir fiziksel bellek mevcut olduğunda programcılar için son derece büyük bir sanal belleğin sağlanmasına izin verir.
 - Programın yürütülmesi için sadece bir kısmının bellekte olması gerekir. Bu nedenle mantıksal adres alanı fiziksel adres alanından çok daha büyük olabilir.
 - Adres alanlarının birkaç process tarafından paylaşılmasına izin verir.
 - Daha verimli process oluşturmaya izin verir.
 - Aynı anda çalışan daha fazla program olmasını sağlar.





Virtual address space

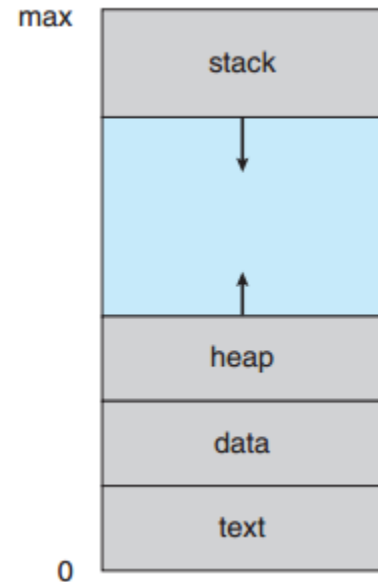
- Bir process'in sanal adres alanı (**virtual address space**), process'in bellekte nasıl depolandığına ilişkin **mantıksal** (veya **sanal**) **görünümü** ifade eder.
 - Tipik olarak bu görünüm, bir process'in belirli bir mantıksal adreste başlamasıdır - örneğin, 0. adres.
 - Bellekteki bir process'in sanal adres alanı şekilde gösterilmiştir - bitişik bellek olarak görünür. Aslında fiziksel bellek, page frame'ler içinde düzenlenmiştir ve bir process'e atanan fiziksel page frame'leri bitişik olmayabilir.
 - Mantıksal page'leri bellekteki fiziksel page frame'lerle eşlemek, bellek yönetim birimine (MMU) bağlıdır.
- Sanal bellek
 - Demand paging
 - Demand segmentationile uygulanabilir.





Virtual address space

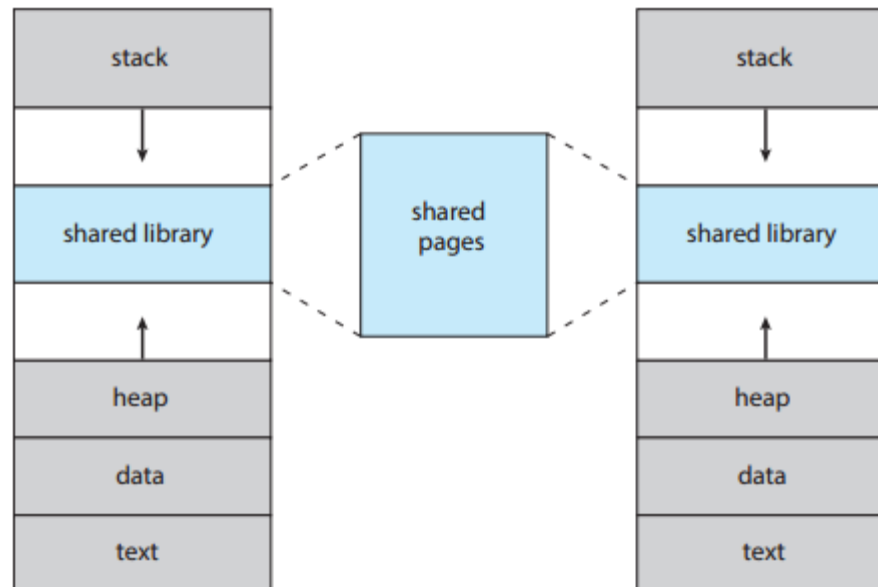
- Genellikle stack'ın max mantıksal adreste başlaması ve heap "yukarı" büyürken stack'ın "aşağı" büyümesi şeklindeki mantıksal adres alanı tasarımı:
 - Adres alanı kullanımını en üst düzeye çıkarır
 - Stack ile heap arasındaki henüz kullanılmamış kısma boşluk (hole) denir.
 - Stack veya heap arasındaki büyük hole sanal adres alanının bir parçasıdır, ancak yalnızca stack veya heap büyüdüğünde gerçek fiziksel page'ler gerektirir.
- Hole'ler içeren sanal adres alanları, **sparse** adres alanları olarak bilinir.
- Sparse bir adres alanı kullanmak faydalıdır çünkü stack veya heap segmentleri büyüdüğçe veya program yürütme sırasında kütüphaneleri dinamik olarak bağlamak istiyorsak hole'ler doldurulabilir.





Virtual address space

- Sanal bellek page paylaşımı yoluyla dosya ve belleğin iki veya daha fazla process tarafından paylaşılmasına da izin verir. Bunun faydaları:
 - Standart C kitaplığı gibi sistem kütüphaneleri, paylaşılan nesnenin sanal bir adres alanına eşleştirilmesi yoluyla birkaç process tarafından şekildeki gibi paylaşılabilir.
 - Sanal bellek ile bir process'ın başka bir process'le bir bellek bölgesini paylaşılmasıyla process'ler arası iletişim yöntemlerinde shared memory'nin uygulanmasını sağlanabilir.
 - Page'ler, process oluşturma sırasında fork() sistem çağrısı ile paylaşılabilir, böylece process oluşturma hızlandırılır.





Sanal Bellek

- Background
- Demand Paging
- Copy-on-Write
- Page Replacement
- Allocation of Frames
- Thrashing

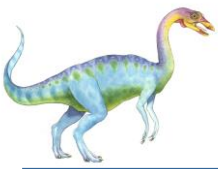




Demand Paging

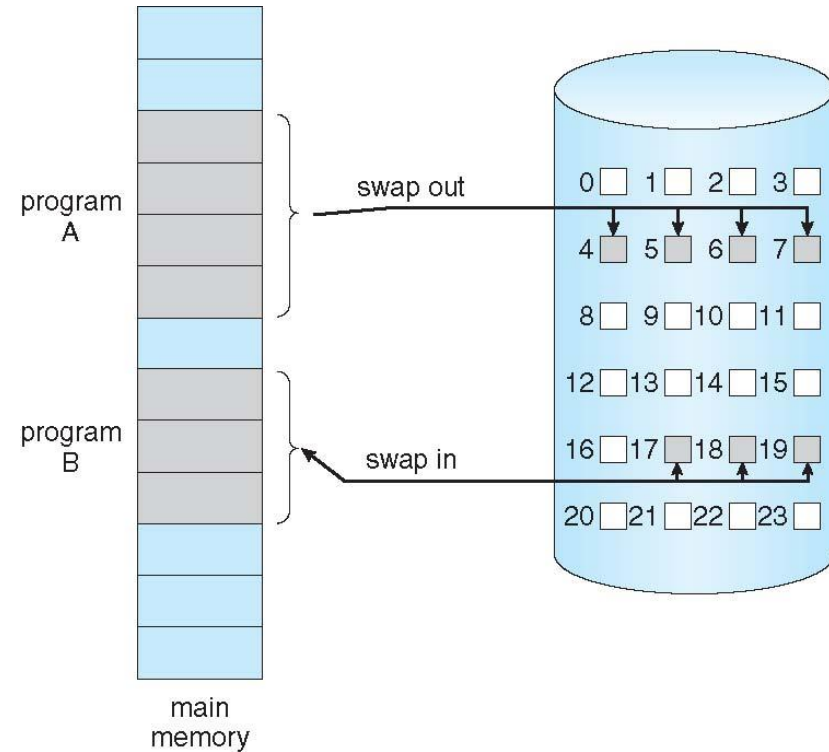
- Diskten belleğe yürütülebilir bir program yüklenirken
- Bir seçenek: tüm programı program yürütme zamanında fiziksel belleğe yüklemektir.
 - Bu yaklaşımla ilgili bir sorun: başlangıçta tüm programa hafızada ihtiyaç duyulmayabilir.
- Diğer bir seçenek, page'leri yalnızca ihtiyaç duyuldukça yüklemektir.
 - Bu teknik, **isteğe bağlı sayfalama** (**demand paging**) olarak bilinir ve genellikle sanal bellek sistemlerinde kullanılır.
 - Daha az I/O'ya ihtiyaç duyar, gereksiz I/O işlemi olmaz.
 - Daha az belleğe ihtiyaç duyulur.
 - Cevap süresi hızlı olur.
 - Daha çok kullanıcının işleri görülebilir.





Demand Paging

- Bir page'e ancak ihtiyaç duyulduğunda referans verilir.
 - Geçersiz referans iptal edilir.
 - Bellekte değilse belleğe getirilir.
- Demand-paging sistemi şekilde gösterildiği gibi, process'lerin diskte bulunduğu swapping yapılan paging sistemine benzer.
- Bir process'i yürütmek istediğimizde, onu hafızaya alırız. Tüm process'i belleğe aktarmak yerine, tembel bir swapper (**lazy swapper**) kullanılır.
 - Tembel bir swapper, o page'e ihtiyaç duyulmadıkça asla belleğe getirmez.
 - Bir swapper, tüm process'leri yönetirken, bir **pager** bir process'in ayrı page'leriyle ilgilenir. Bu nedenle, demand paging ile bağlantılı olarak "swapper" yerine "**pager**" terimi kullanılır.





Temel Kavramlar

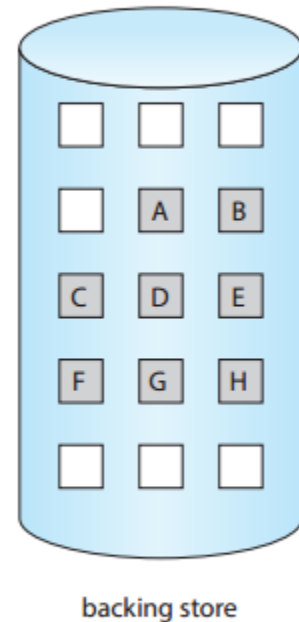
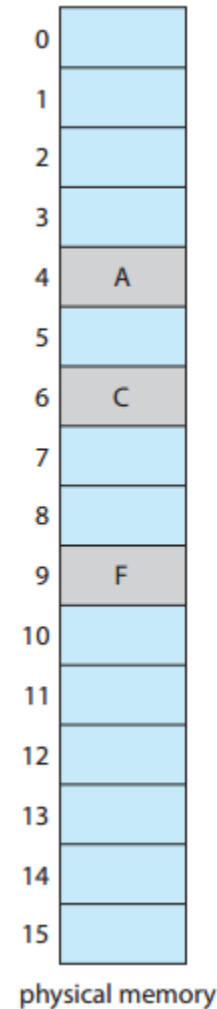
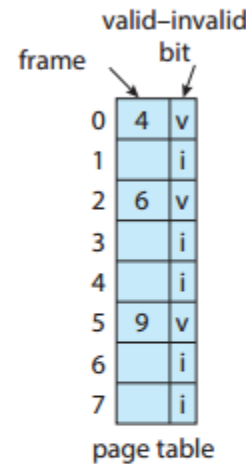
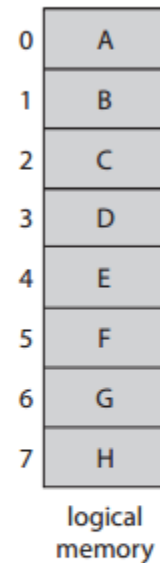
- Bir process'in **swap in** yapılması gerektiğinde, **pager** hangi page'lerin kullanılacağını tahmin eder.
 - Tüm process için swapping yapmak yerine, **pager** yalnızca bu page'leri belleğe getirir.
- Bir process yürütülürken bazı page'ler bellekte, bazıları ise diskte olacaktır. Bu nedenle, ikisini birbirinden ayırmak için bir tür donanım desteğine ihtiyacımız vardır.
 - Önceki bölümde açıklanan valid-invalid bit şeması bu amaçla kullanılabilir.
- Gerekli page'ler zaten bellekte yerleşikse (**memory resident**)
 - demand-paging olmayan sistemle aynıdır.
- Page gerekliyse ve bellekte yerleşik değilse
 - Program davranışını değiştirmeden
 - Programcının kodu değiştirmesine gerek kalmadanpage'in diskten algılanması ve belleğe yüklenmesi gerekir.





Valid-Invalid Bit

- Page tablosunun her girişi valid–invalid biti ile ilişkilendirilir.
(**v** $\Rightarrow$ in-memory – **memory resident**, **i** $\Rightarrow$ not-in-memory)
- Başlangıçta tüm girişler için valid–invalid biti = **i**
- Page tablosunun örnek bir görüntüsü şekilde verilmiştir.



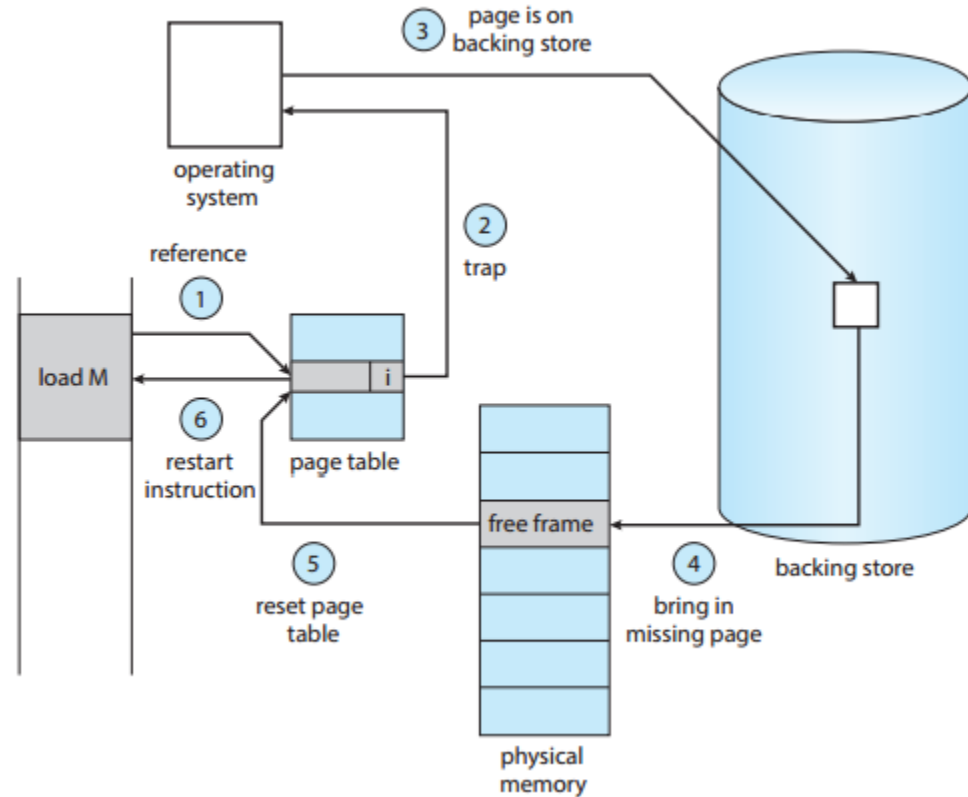


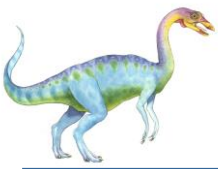
Page Hatasını İşleme Prosedürü

Process, belleğe henüz getirilmemiş bir page'e erişmeye çalıştığında invalid bir page'e erişim, page hatasına neden olur.

Page hatasını ele alma prosedürü şöyledir:

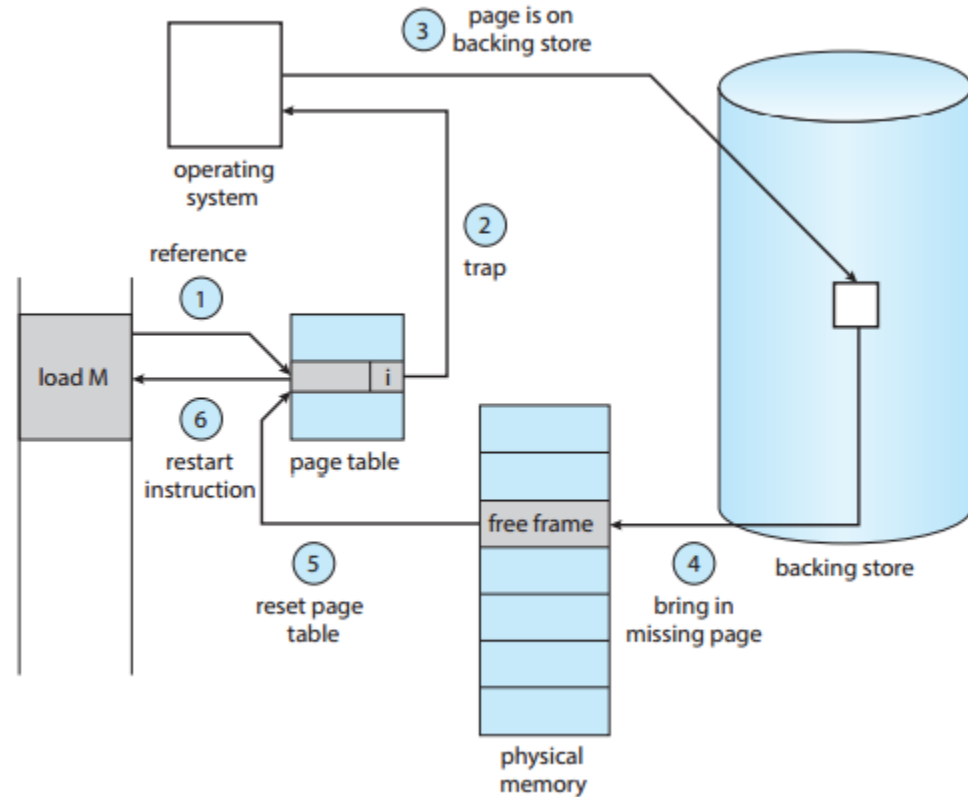
1. Bu process için referansın geçerli mi, yoksa geçersiz bir bellek erişimi mi olduğunu belirlemek için dahili bir tablo kontrol edilir.
2. Referans **geçersizse**, process'i sonlandırırız. Eğer **geçerliyse** ve page henüz getirilmediyse paging yapılır.
3. Boş bir frame bulunur.





Page Hatasını İşleme Prosedürü

4. İstenilen page'i yeni tahsis edilen frame'e okumak için ikincil bir depolama işlemi schedule ederiz.
5. Depolama okuması tamamlandığında, process'te tutulan dahili tablo ve page'in artık bellekte olduğunu belirtmek için page tablosu değiştirilir.
Validation bit = = **V**
6. Page hatası nedeniyle kesintiye uğrayan komutu yeniden başlatılır. Process artık page'e sanki her zaman bellekteymiş gibi erişebilir.





Demand Paging

- En ekstra durum – page'lerinden hiçbiri belleğe henüz getirilmemiş bir process'i yürütmeye başlatmaktır.
 - İşletim sistemi, instruction pointer'ı process'in bellekte yerleşik olmayan page'deki ilk komutuna ayarladığında, process anında page hatası verir.
 - Bu page belleğe alındıktan sonra process yürütülmeye devam eder, **ihtiyaç duyduğu her page** belleğe alınana kadar **gerektiği kadar** page hatası verir.
 - ▶ Bu noktadan itibaren daha fazla hata olmadan çalışabilir. Bu şema **pure demand paging**'tir: bir page gerekmedikçe asla belleğe getirilmez.
- Aslında, belirli bir komut birden çok page'e erişebilir ve birden çok page hatası olabilir. Bu durum, kabul edilemez sistem performansına neden olur.
 - Ancak çalışan process'lerin analizi, bu davranışın pek olası olmadığını göstermektedir.
 - Programlar lokal referans bölgesine (**locality of reference**) sahip olma eğilimindedir ve bu da demand paging'in makul performansla sonuçlanmasını sağlar.





Demand Paging

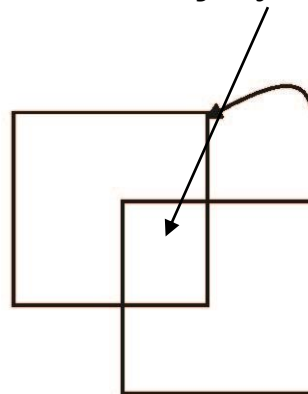
- Demand paging destekleyen donanım, paging ve swapping donanımıyla aynıdır:
 - **Page tablosu:** Bu tablo, valid-invalid bir biti veya özel bir koruma biti ile geçersiz bir girişi belirtmeyi sağlar.
 - **İkincil bellek:** Bu hafıza, ana hafızada bulunmayan page'leri diskte özel bir bölgede tutar. İkincil bellek genellikle yüksek hızlı bir disk, **swap device** olarak bilinir, bu amaçla kullanılan disk bölümüne de **swap space** denir.





Instruction Restrat

- Demand paging için çok önemli bir gereksinim, bir page hatasından sonra herhangi bir komutun yeniden başlatılabilmesidir.
- Page hatası olduğunda interrupt olan process'in durumunu kaydettiğimiz için, istenen page belleğe getirilip erişilebilir olduğunda process'i **tam olarak kaldığı yerden** yeniden başlatabilmeliyiz.
- Demand paging'de page hatasının ele alınmasında **en büyük zorluk**, bir komut birkaç farklı konuma erişip bunları değiştirdiğinde ortaya çıkar.
 - Örneğin bir komut bir konumdan diğerine bir bloğu taşıyor olsun.
 - Bloklardan biri (kaynak veya hedef) bir page sınırını aşarsa, taşıma kısmen tamamlandıktan sonra bir page hatası oluşabilir.
 - Ayrıca, kaynak ve hedef blokları çakışırsa, kaynak blok değiştirilmiş olabilir, bu durumda komutu basitçe yeniden başlatamayız.





Instruction Reformat

- Bu problem iki farklı şekilde çözülebilir.
 - Birinci çözümde, bir mikrokod her iki bloğun her iki ucunu da hesaplar ve bunlara erişmeye çalışır.
 - ▶ Bir page hatası oluşacaksa, herhangi bir değişiklik yapılmadan önce bu adımda gerçekleşecektir.
 - ▶ Page hatası yoksa taşınma daha sonra gerçekleşebilir.
 - Diğer çözüm, üzerine yazılan konumların değerlerini tutmak için geçici register'ler kullanılır.
 - ▶ Bir page hatası varsa, hata oluşmadan önceki tüm eski değerler belleğe yazılır.
 - ▶ Bu eylem, belleği komut başlamadan önceki durumuna geri yükler, böylece komut tekrarlanabilir.





Free-Frame List

- Bir page hatası oluştuğunda, işletim sistemi istenen page'i diskten ana belleğe getirmelidir.
- Page hatalarını çözmek için çoğu işletim sistemi şekildeki gibi bir **free-frame list** tutar: bu tür istekleri karşılamak için bir boş frame havuzudur.



- İşletim sistemi tipik olarak, zero-fill-on-demand olarak bilinen bir teknik kullanarak boş/kullanılabilir frame'ler tahsis eder.
- Zero-fill-on-demand frame'ler tahsis edilmeden önce sıfırlanır ve böylece önceki içerikleri silinir.
- Bir sistem başladığında, kullanılabilir tüm bellek free-frame listesine yerleştirilir.





Demand Paging En Kötü Durum Aşamaları

- Bir page hatası en kötü durumda aşağıdaki aşamaların oluşmasına neden olur:
 1. İşletim sisteminde hata oluşur
 2. Process'le ilgili register ve process'in durumu kaydedilir.
 3. Interrupt'ın bir page hatası olduğunu belirlenir
 4. Page referansının geçerli olup olmadığını kontrol edilir ve page'in diskteki konumunu belirlenir.
 5. Diskten bellekte boş bir frame'e okuma adımları:
 - a) Okuma talebine hizmet verilinceye kadar bu aygıt için kuyrukta beklenir
 - b) Aygıt içinde arama ve/veya gecikme süresi beklenir.
 - c) Page boş bir frame'e aktarılmaya başlanır





Demand Paging En Kötü Durum Aşamaları

6. Diskten belleğe okuma işleminin tamamlanması beklenirken CPU'ya başka bir process işlerini yürütmek üzere geçer.
7. I/O tamamlanınca disk I/O alt sisteminden bir interrupt alır
8. Bekleme esnasında CPU'ya geçen process'in register'leri ve durumu kaydedilir (6. adım yürütülmüşse).
9. Interrupt'ın diskten olduğu belirlenir.
10. İstenilen page'in artık bellekte olduğunu göstermek için page tablosu ve diğer tablolar güncellenir.
11. CPU'nun bu process'e yeniden tahsis edilmesi beklenir.
12. Process'in en son kaldığı noktada kaydedilen register'leri, process durumu ve yeni page tablosu geri yüklenir ve ardından interrupt edilen komut kaldığı yerden devam ettirilir.





Demand Paging Performansı

- Demand paging, bir bilgisayar sisteminin performansını önemli ölçüde etkileyebilir.
- Nedenini görmek için, deaman paged bellek için **effective acces time** hesaplayalım.
- Demand page hatasının en kötü durum aşamalarının tümü her zaman oluşmaz.
 - Örneğin, 6. adımda, I/O gerçekleşirken CPU'nun başka bir işleme tahsis edildiğini varsayıyoruz.
 - Bu düzenleme, multiprogramming'in CPU kullanımını sürdürmesine izin verir, ancak I/O transferi tamamlandığında page hatası servis rutinini sürdürmek için ek süre gerektirir.
- Her durumda, page hatası servis süresinin üç ana kısmı vardır:
 1. Page hatası interrupt'ın ele alınması.
 2. Page'in okunması – en çok zaman kaybı burada olur.
 3. Process'in yeniden başlatılması.





Demand Paging Performansı

- p , page hata oranı olsun, $0 \leq p \leq 1$.
 - $p = 0$ ise page hatası yoktur.
 - $p = 1$ ise her page referansı bir hatadır.
- Effective Access Time (EAT) şöyle hesaplanır:
$$\text{EAT} = (1 - p) \times [\text{bellek erişim süresi}] + p \times [\text{page hatası süresi}]$$





Demand Paging Örnek

- Bellek erişim süresi = 200 nanosaniye
- Ortalama page-hata servis süresi = 8 milisaniye
- $EAT = (1 - p) \times 200 + p \times 8,000,000 = 200 + p \times 7,999,800$
- EAT'in page hatası oranıyla doğru orantılı olduğu görülmektedir.
- 1000 erişimden biri bir page hatasına neden olursa
 $EAT = 200 + 0,001 \times 7,999,800 = 8.2$ mikrosaniye olur.
Demand paging nedeniyle bilgisayar 40 kat yavaşlayacaktır!
(200 nanosaniye*40=8 mikrosaniye)
- Performans düşüşünün yüzde 10'dan az olmasını istiyorsak, page hatası olasılığını aşağıdaki seviyede tutmamız gerekir:
 - $220 > 200 + 7,999,800 \times p$
 $20 > 7,999,800 \times p$
 - $p < .0000025$
 - < her 400.000 bellek erişiminde bir page hatası olmalı
- Özetle, bir demand paging sisteminde page hatası oranını düşük tutmak önemlidir. Aksi takdirde, EAT artar ve process'in yürütülmesi önemli ölçüde yavaşlar.





Demand Paging Optimizations

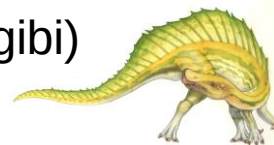
- Demand paging'in ele alınması gereken bir diğer yönü, swap space'in ele alınması ve genel kullanımıdır.
- Swap space I/O, çok daha büyük bloklarda tahsis edildiğinden ve dosya aramaları (file lookups) ve dolaylı tahsis (indirect allocation) yöntemleri kullanılmadığından dosya sistemi I/O'dan daha hızlıdır.





Demand Paging Optimizations

- Sistemin daha iyi bir paging verimi elde etmesi için
 1. Seçenek: process **başlangıcında** bir dosya görüntüsünün tamamını swap space'e kopyalamak ve ardından **swap space'den demand paging** gerçekleştirmektir.
 - Bu yaklaşımın bariz dezavantajı, program başlangıcında dosya görüntüsünün kopyalanmasıdır.
 - BSD Unix'in eski sürümlerinde kullanılmıştır.
 2. Seçenek: **başlangıçta dosya sisteminden demand page** yapmaktır,
 - Ancak page'ler bellekte değiştirildikçe tekrar dosya sistemine değil swap space'e yazılır.
 - Linux ve Windows da dahil çeşitli işletim sistemleri tarafından uygulanır.
- Mobil işletim sistemleri
 - Swapping'i desteklemez
 - Bunun yerine, bu sistemler dosya sisteminden page talep eder ve bellek kısıtlanırsa uygulamalardan salt okunur page'leri (kod gibi) geri alır.





Sanal Bellek

- Background
- Demand Paging
- **Copy-on-Write**
- Page Replacement
- Allocation of Frames
- Thrashing





Copy-on-Write

- `fork()` sistem çağrısını kullanarak process oluşturma, başlangıçta page paylaşımına benzer bir teknik kullanarak demand paging ihtiyacını atlayabilir.
- Bu teknik, hızlı process oluşturma sağlar ve yeni oluşturulan process'e atanması gereken yeni page'lerin sayısını en aza indirir.
- `fork()` sistem çağrısı parent process'in kopyası olan bir child process oluştuyordu. Geleneksel olarak `fork()`, parent'in child için adres alanının bir kopyasını oluşturarak, parent'e ait page'leri kopyalayarak çalışıyordu.





Copy-on-Write

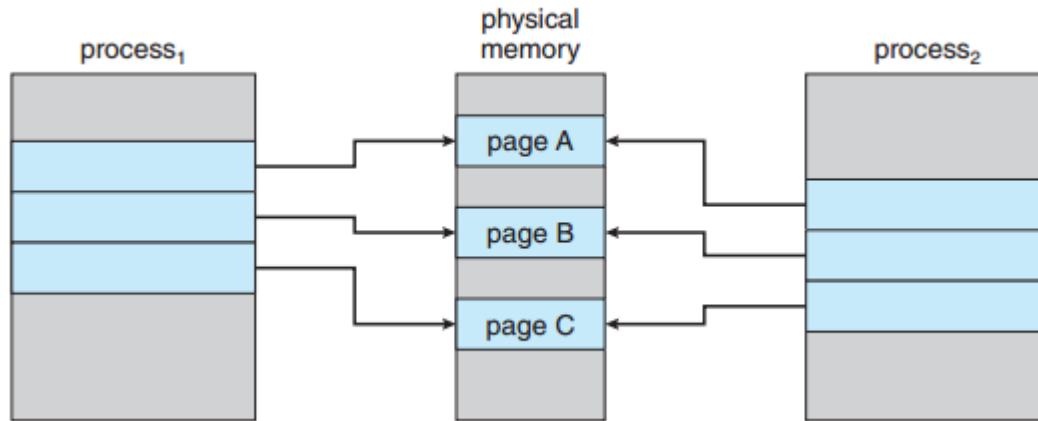
- Ancak, birçok child process'in oluşturulduktan hemen sonra exec() sistem çağrısını çağırdığı düşünüldüğünde, parent öğenin adres alanının kopyalanması gereksiz olabilir.
 - Bunun yerine, parent ve child process'lerin başlangıçta aynı page'leri paylaşmasına izin vererek çalışan **Copy-on-Write** (COW) olarak bilinen bir teknik kullanılabilir.
 - Bu paylaşılan page'ler, copy-on-write page'ler olarak işaretlenir, daha sonra process'lerden biri paylaşılan bir page'e bir şey yazacak olursa paylaşılan page'in bir kopyası oluşturulur.



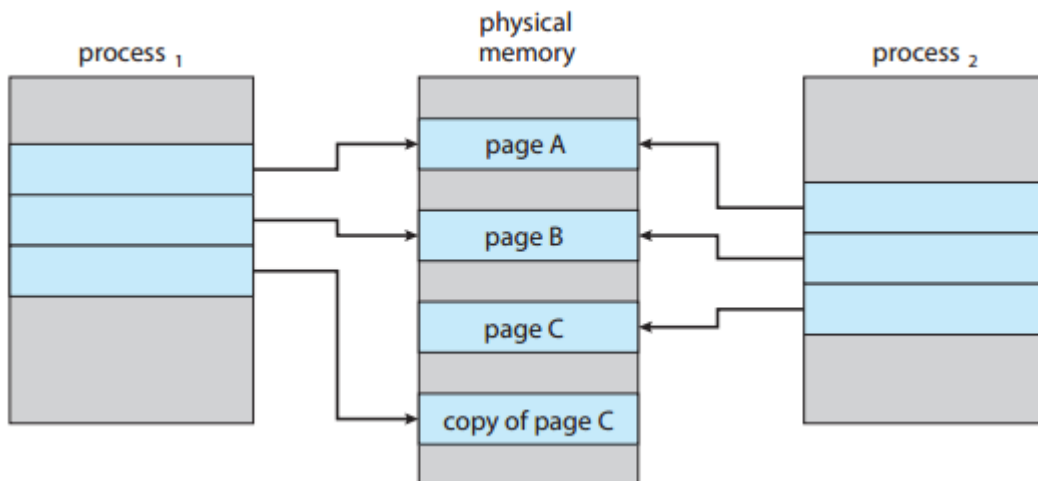


Copy-on-Write

- process 1'in page C'yi değiştirmeden önceki ve sonraki fiziksel belleğin içeriği şekillerde gösterilmiştir.



p1 process'i page C'yi değiştirmeden önce



p1 process'i page C'yi değiştirdikten sonra





Sanal Bellek

- Background
- Demand Paging
- Copy-on-Write
- Page Replacement
- Allocation of Frames
- Thrashing





Sanal Bellek

- Page Replacement
 - Basic Page Replacement
 - First-In-First-Out Algorithm
 - Optimal Page Replacement Algorithm
 - Least Recently Used Algorithm
 - LRU Approximation Algorithms
 - ▶ Additional-Reference-Bits Algorithm
 - ▶ Second-Chance Algorithm
 - ▶ Enhanced Second-Chance Algorithm
 - Page-Buffering Algorithms
 - Applications and Page Replacement

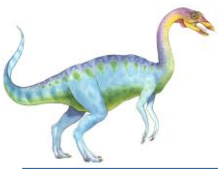




Page Replacement

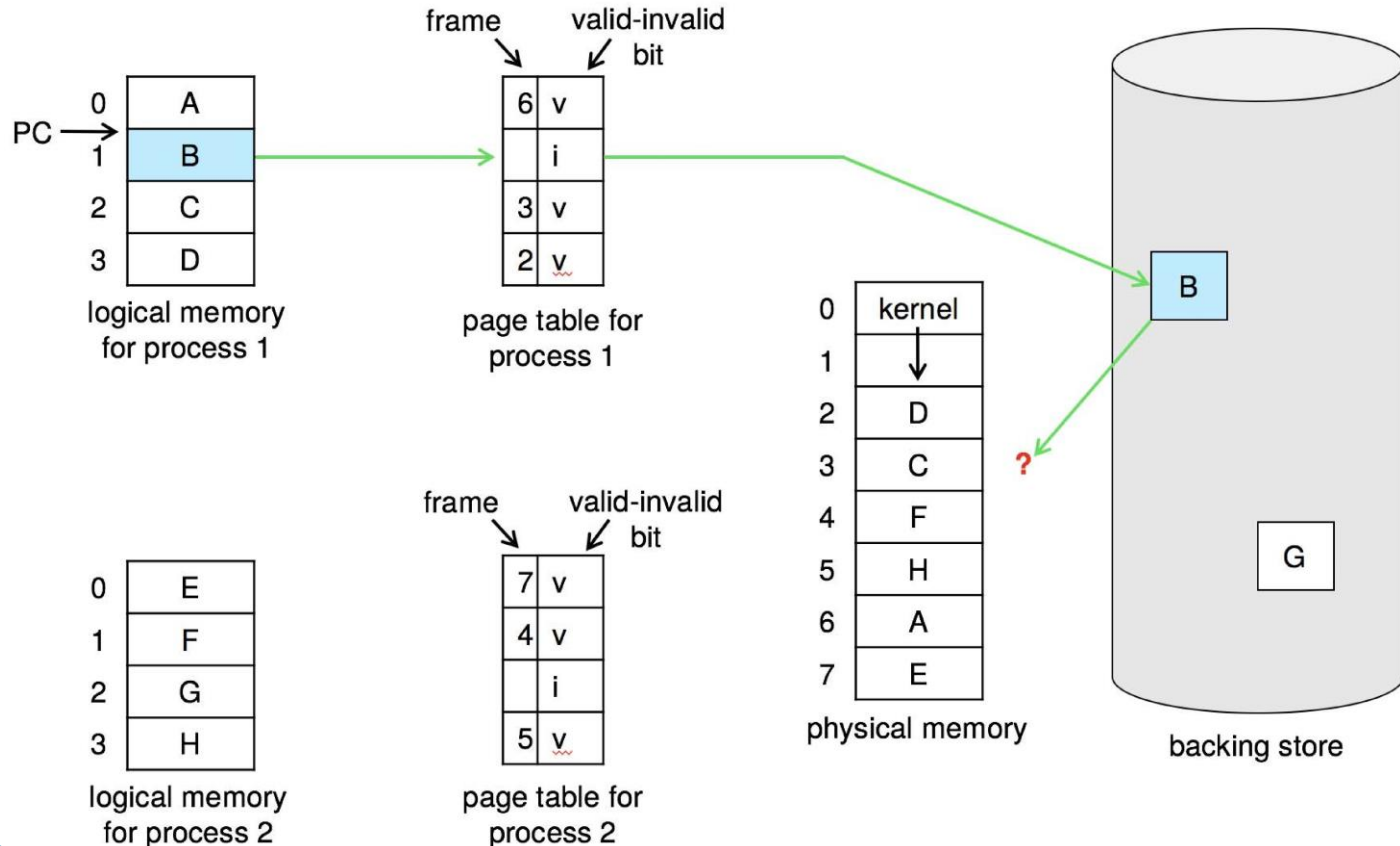
- Multiprogramming derecesinin arttırılması bellekteki process sayısının artması demektir.
- Multiprogramming derecesi arttığında bellekte bulunan process'lerin ihtiyaç duyduğu frame sayısı belleğin kapasitesini aşabilir, bu duruma belleğin aşırı tahsisi (**over-allocating**) denir.
- Ayrıca programların yanı sıra kernel ve I/O buffer'leri de belleği kullanmaktadır.
- Yukarıda bahsedilen durumlar bellek tahsisi algoritmalarındaki yükü artırabilir.
- I/O'ya ne kadar bellek ayrılacağına ve page'lerin ne kadar programlanacağına karar vermek önemli bir problemdir.





Boş Frame Yoksa Ne Olacak?

- Belleğin over-allocation durumu şöyle kendini gösterir:
 - Bir process yürütülürken bir page hatası oluşur. İşletim sistemi, istenen page'ın diskte nerede bulunduğunu belirler.
 - Ancak daha sonra boş frame listesinde boş frame olmadığını görür; tüm bellek kullanımdadır.





Boş Frame Yoksa Ne Olacak?

- Çoğu işletim sistemi over-allocation için **page swapping** ve **page replacement** bilikte kullanır.
- Page replacement tekniğinde hiçbir frame boş değilse, şu anda kullanılmayan bir frame bulunur ve frame serbest bırakılır.
 - Bir frame şöyle serbest bırakılır: frame'in içeriği swap space'e yazılır, page'in artık bellekte olmadığını göstermek için page tablosu (ve diğer tüm tabloları) değiştirilir.
 - Artık process'in hata aldığı page'i yerleştirmek için serbest bırakılan frame kullanılabilir.





Page Replacement

- **Page replacement, demand paging için temeldir.** Mantıksal bellek ile fiziksel bellek arasındaki ayrımı tamamlar.
 - Bu mekanizma ile programcılar için küçük bir fiziksel bellekte oldukça büyük bir sanal bellek sağlanabilir.
- **Demand paging olmadan,** mantıksal adresler fiziksel adreslerle eşlenir ve iki adres kümesi farklı olabilir. Bununla birlikte, bir process'in tüm page'leri yine de fiziksel bellekte olmalıdır.
- **Demand paging ile** mantıksal adres alanının boyutu artık fiziksel bellek tarafından kısıtlanmaz.





Page Replacement

- 20 page'lik bir process'imiz varsa, bunu yalnızca **demand paging** kullanarak ve gerektiğinde boş bir frame bulmak için bir **page replacement** kullanarak 10 frame içinde yürütebiliriz.
- Bir page yer değiştirilecekse içeriği ikincil depolamaya kopyalanır. Bu page'e daha sonra yapılan bir referans page hatasına neden olacaktır.
 - O zaman page, belki de process'teki başka bir page'in yerini alarak belleğe geri getirilecektir.





Basic Page Replacement

- Page hatası servis rutini, page replacement'i içerecek şekilde değiştirilerek aşırı bellek tahsisi (**over-allocation of memory**) önlenir:

1. Diskte istenen page'ın konumunu bulunur

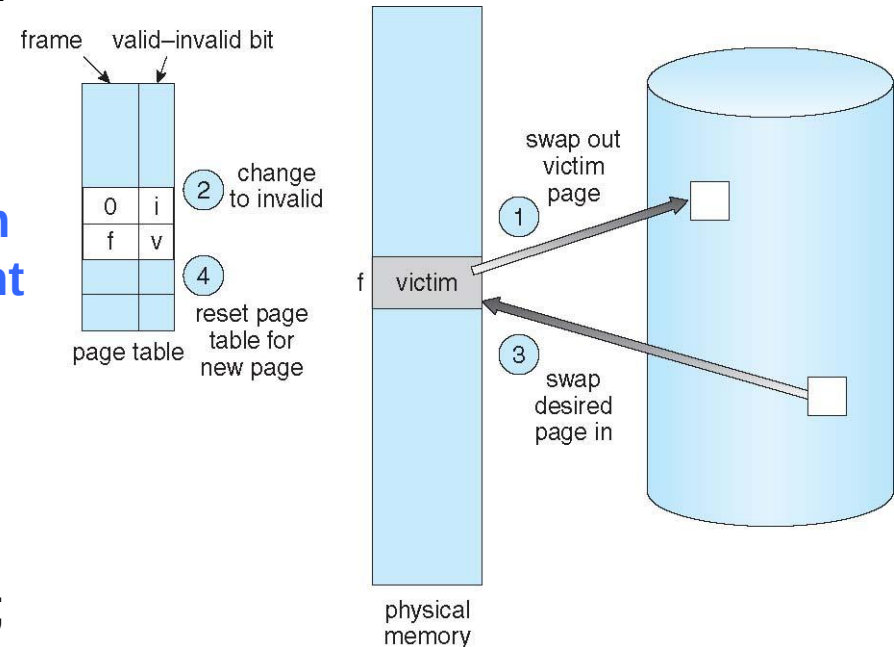
2. Daha sonra boş bir frame bulunur:

- Boş bir frame varsa kullanılır
- Boş frame yoksa, kurban frame'i (**victim frame**) seçmek için bir **page replacement algoritması** kullanılır.

- **Victim frame** diske yazılır (gerekirse); buna göre page ve frame tabloları değiştirilir (page out)

3. İstenilen page (yeni) boş frame'e getirilir; page ve frame tabloları güncellenir (page in)

4. Process page hatasının oluştuğu yerden yürütülmeye devam eder.





Page Replacement

- Page hata servis rutininde hiçbir frame boş değilse, iki page aktarımı (biri page out ve biri page in için) gerekir. Bu durum, page hatası servis süresini ikiye katlar ve buna göre EAT'yi artırır.
- Bir **modify bit** kullanarak bu ek yükü azaltabiliriz - yalnızca değiştirilmiş page'ler diske yazılır. Değiştirilmemiş halde bellekte duran page'i diske yazmamıza gerek yoktur: zaten aynısı diskte dir.





Page and Frame Replacement Algorithms

- **Demand paging** uygulamak için iki ana sorunu çözmeliyiz:
 1. Bellekte birden fazla process varsa, her bir process'e kaç frame tahsis edeceğimize karar vermeliyiz.
 - ▶ Bunun için **Frame-allocation algoritması** geliştirmemiz gerekir.
 2. Page replacement gerektiğinde, değiştirilecek frame'leri seçmemiz gerekir.
 - ▶ Bunun için **Page-replacement algoritması** geliştirmemiz gerekir.
- Birçok farklı page-replacement algoritması vardır. Bir page-replacement algoritmasını neye göre seçeriz?
 - Genel olarak istenen, page hatası oranının en düşük olmasıdır.





Page and Frame Replacement Algorithms

- Bir replacement algoritması, belirli bir bellek referans string'inde çalıştırılıp page hatalarının sayısının hesaplanmasıyla değerlendirilir.
- Değerlendirme yapılırken:
 - String sadece page numaralarıdır, tam adresler değildir.
 - Bir p page'ine bir referansımız varsa, **hemen ardından** p page'ine yapılan herhangi bir referans asla bir page hatasına neden olmaz. P page'i ilk referanstan sonra bellekte olacaktır, bu nedenle hemen sonraki referanslarda hata olmayacaktır.
 - Sonuçlar, mevcut frame sayısına bağlıdır.
- Sonraki slaytlarda verilecek örneklerde, referans verilen page numaralarının referans string'i şu şekildedir:

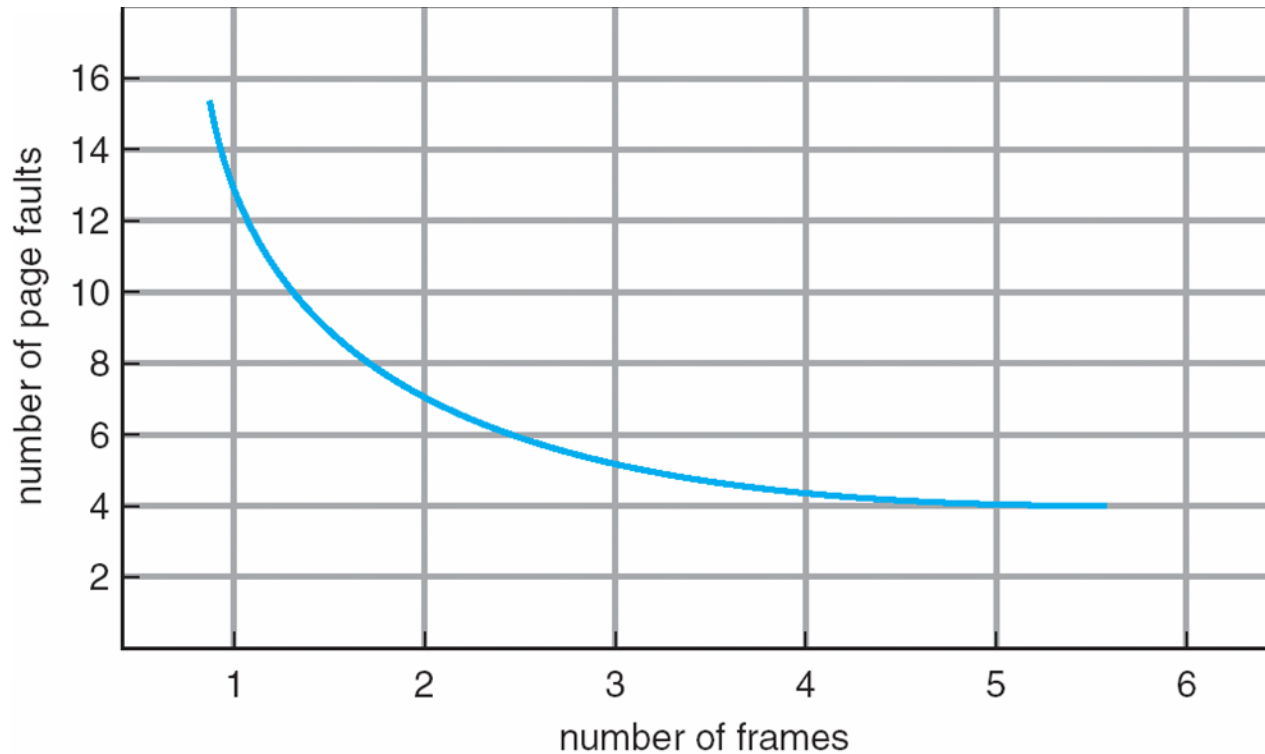
7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1





Frame Sayısına Karşı Page Hatalarının Grafiği

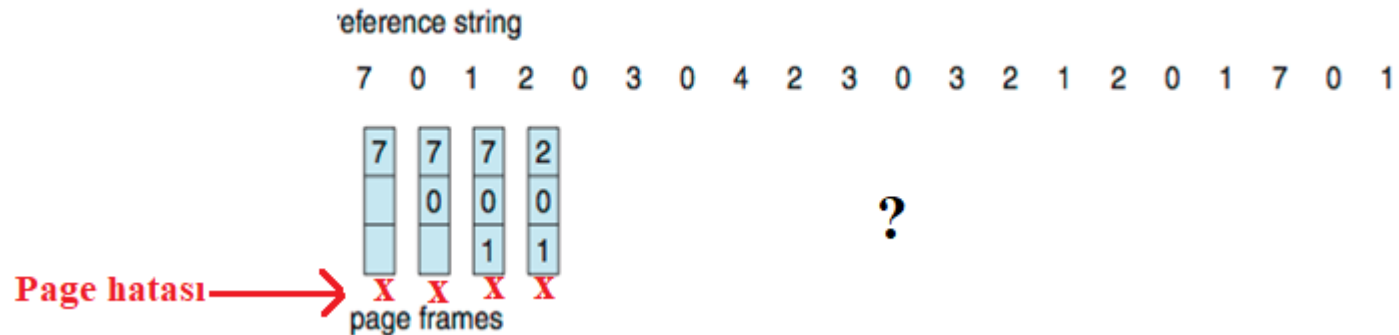
- Mevcut frame sayısı arttıkça, page hatalarının sayısı azalır.





First-In-First-Out (FIFO) Algorithm

- FIFO algoritmasında ilk gelen frame atılarak yeni gelen buraya yazılır.
- Şekildeki örnek referans string için FIFO algoritmasına göre page'lerin yerleşimi nasıl olur, kaç page hatası olur?
 - Proses başına bir seferde **3 page** bellekte olabilir.





First-In-First-Out (FIFO) Algorithm

- FIFO algoritmasında ilk gelen frame atılarak yeni gelen buraya yazılır.
- Şekildeki örnekte process için bellekte 3 frame ayrılmıştır. (Process başına bir seferde 3 page bellekte olabilir)

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2	2	2	4	4	4	0	0	0	7	7	7
	0	0	0	3	3	3	2	2	2	1	1	1	0	0
		1	1	1	0	0	0	3	3	3	2	2	2	1

page frames

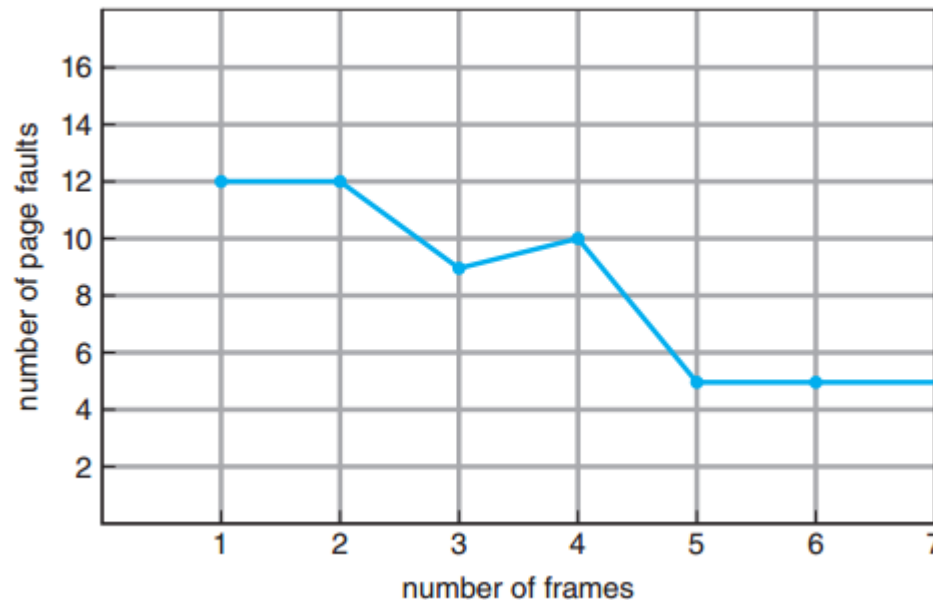
- Örnek referans dizimiz için, üç frame başlangıçta boştur. İlk üç referans (çağrı) (7, 0, 1) page hatalarına neden olur ve bu boş frame'lere getirilir.
- Daha sonra page 2, page 7'nin yerini alır, çünkü page 7 ilk sırada getirilmiştir. Ardından page 0 sonraki çağrılacak page'tir, zaten bellekte olduğundan bu referans için page hatası olmaz.
- Page 3'e yapılan ilk referans, artık ilk sırada olduğu için page 0'ın değiştirilmesiyle sonuçlanır. Bu değiştirme nedeniyle, bir sonraki page 0 referansı hatalı olacaktır.*
- Page 1, daha sonra page 0 ile değiştirilir. Bu işlem Şekilde gösterildiği gibi devam eder. Her hata oluşuktan sonra hangi page'lerin frameler içinde olduğu gösterilmiştir. Toplamda 15 page hatası oluşur.





First-In-First-Out (FIFO) Algorithm

- Bir FIFO page replacement algoritmasıyla oluşabilecek sorunları göstermek için şöyle bir referans string'i olsun: 1,2,3,4,1,2,5,1,2,3,4,5
- Şekilde farklı frame sayılarına göre bu referans string için page hatalarının eğrisi gösterilmiştir.
- 4 frame için hata sayısı (10), 3 frame için hata sayısından (9) daha fazla olmuştur.
 - Bu en beklenmedik sonuç, Belady anormalliği (**Belady' s Anomaly**) olarak bilinir: bazı page replacement algoritmaları için tahsis edilen frame'lerin sayısı arttıkça page hatası oranı artabilir.





Optimal Page Replacement Algorithm

- **Optimal Page Replacement Algorithm (OPT)** - tüm algoritmalar arasında en düşük page hatası oranına sahiptir, Belady anormalliğinden asla etkilenmez.
- Algoritmada, uzun süre kullanılmayacak olan page ile yer değiştirme yapılır.
- Bu algoritmaya göre bir process için aşağıda verilen referans string'ine göre page'lerin yerleşimi nasıl olur? (Process başına bir seferde 3 page bellekte olabilir)
- Bu algoritmada kaç page hatası olur?

reference string

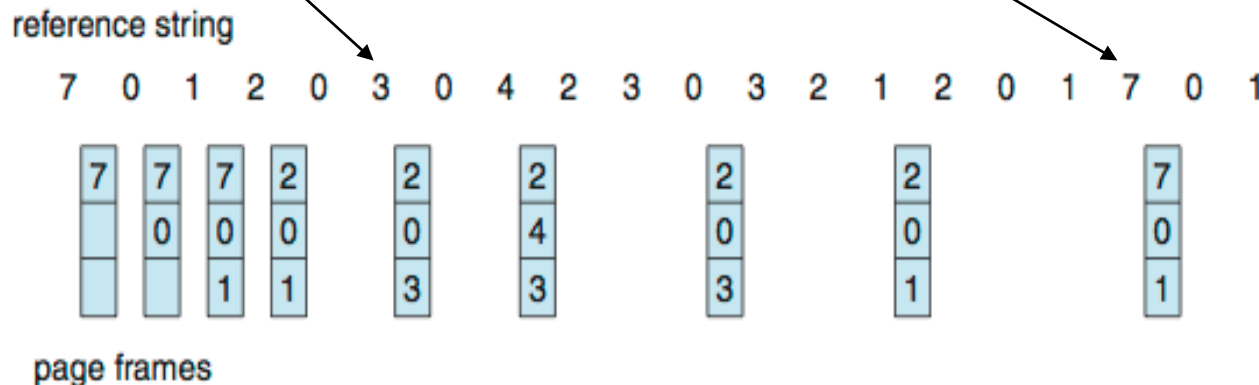
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1





Optimal Page Replacement Algorithm

- **Optimal Page Replacement Algorithm (OPT)** - tüm algoritmalar arasında en düşük page hatası oranına sahiptir, Belady anormalliğinden asla etkilenmez.
- Algoritmada, uzun süre kullanılmayacak olan page ile yer değiştirme yapılır.
- Örnekte (7,0,1) yerleştikten sonra page 2 referansı page 7'nin yerine geçer; çünkü page 7, 18. referansa kadar kullanılmayacaktır, oysa page 0 ve page 1 daha önce kullanılacaktır.
- Page 3'e yapılan atıf, page 1'in yerini alır, çünkü page 1, bellekteki üç page içinden tekrar referans alınacak sonuncusu page'dir.
- Bu algoritmada 9 page hatası olur.





Optimal Page Replacement Algorithm

- Optimal page replacement algoritmasının uygulanması zordur, çünkü referans string'inin önceden bilinmesini gerektirir.
- Bu algoritma esas olarak yeni önerilecek algoritmaları değerlendirirken karşılaştırma çalışmaları için kullanılır.





Least Recently Used (LRU) Algorithm

- Gelecekteki ziyade geçmiş bilgileri kullanır
- En uzun süre kullanılmayan page'i değiştirir.
- LRU algoritması, her page'le o page'in son kullanım zamanını ilişkilendirir.
- Bu algoritmaya göre bir process için aşağıda verilen referans string'ine göre page'lerin yerleşimi nasıl olur? (Process başına bir seferde 3 page bellekte olabilir)
- Bu algoritmada kaç page hatası olur?

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1





Least Recently Used (LRU) Algorithm

- Gelecekteki ziyade geçmiş bilgileri kullanır
- En uzun süre kullanılmayan page'i değiştirir.
- LRU algoritması, her page'le o page'in son kullanım zamanını ilişkilendirir

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0		1		1		1
	0	0	0		0		0	0	3	3		3		0		0
		1	1		3		3	2	2	2		2		2		7

page frames

- Page 4'e referans gerçekleştiğinde, algoritma bellekteki üç frame'den page 2'nin uzun süre kullanılmadığını görür. Bu nedenle algoritma, page 2'nin sonraki adımda kullanılacağını bilmeden page 2'nin yerini page 4'e verir.
- Örnekte 12 page hatası olur—FIFO'dan iyi fakat OPT'dan kötüdür.
- Genellikle iyi sonuç veren bir algoritmadır sıklıkla kullanılır.





LRU Algorithm (Cont.)

- Bu algorithmda en büyük sorun nasıl uygulanacağıdır. Bir LRU algoritması, önemli bir donanım yardımı gerektirebilir.
- Sorun, son kullanım zamanına göre tanımlanan frame'ler için bir sıra belirlemektir. İki farklı şekilde uygulanabilir:
 1. Sayaç gerçekleştirimi (Counter implementation)
 - Her page girişinin bir sayacı vardır; page'e bu girişle her başvurulduğunda, saat sayaca kopyalanır.
 - Bir page'in değiştirilmesi gerektiğinde, en küçük değeri bulmak için sayaçlara bakılır. Tabloda arama gerekir.





LRU Algorithm (Cont.)

2. Yığın Gerçekleştirimi (Stack implementation)

- Bir page'e başvurulduğunda o page yığıttan kaldırılıp yığının en üstüne tekrar konur.
- Bu şekilde, en son kullanılan page her zaman yığının en üstünde, en az kullanılan page ise her zaman en altında yer alır.

reference string

4 7 0 7 1 0 1 2 1 2 7 1 2

2
1
0
7
4

stack
before
a

7
2
1
0
4

stack
after
b

a b





LRU Algorithm (Cont.)

- Girişlerin yığıtın ortasından kaldırılması gerektiğinden, bu yaklaşımı bir head pointer ve bir tail pointer ile double linked list kullanarak uygulamak en iyisidir.
- Önceki örnekte bir page'i kaldırıp yığıtın en üstüne koymak, en kötü ihtimalle 6 pointer'ı değiştirmeyi gerektirir.
- Her güncelleme biraz daha pahalıdır, ancak page değiştirme için aramaya gerek yoktur; kuyruk pointer'ı, en uzun süre kullanılmayan page'i yığıtın en altında işaret eder.
- Optimal algoritma gibi, LRU algoritması da Belady anormalliğinden etkilenmez.
 - OPT ve LRU algoritmaları, Belady'nin anormalliğini asla göstermeyen, yığıt algoritmaları (**stack algorithms**) adı verilen bir page replacement algoritmaları sınıfına aittir.





Sanal Bellek

- Page Replacement
 - Basic Page Replacement
 - First-In-First-Out Algorithm
 - Optimal Page Replacement Algorithm
 - Least Recently Used Algorithm
 - LRU Approximation Algorithms
 - ▶ Additional-Reference-Bits Algorithm
 - ▶ Second-Chance Algorithm
 - ▶ Enhanced Second-Chance Algorithm
 - Page-Buffering Algorithms
 - Applications and Page Replacement





LRU Approximation Algorithms

- Pek çok bilgisayar sistemi, gerçek LRU page replacement için yeterli donanım desteği sağlamaz.
- Bununla birlikte, birçok sistem bir **referans biti** şeklinde bir miktar yardım sağlar.
- Referans bitleri, page tablosundaki her girişle ilişkilendirilir.
- Başlangıç değeri 0 olan page'in referans biti, o page'e her referans verildiğinde donanım tarafından 1 olarak set edilir.
- Bir süre sonra **kullanım sırasını bilmesek de** referans bitlerini inceleyerek yakın zamanda hangi page'lerin kullanıldığını ve hangilerinin kullanılmadığını belirleyebiliriz.
- Bu bilgi, LRU replacement'ı andıran (**approximation**) birçok page replacement algoritmasının temelidir.





LRU Approximation Algorithms

- Additional-Reference-Bits Algorithm
- Second-Chance Algorithm
- Enhanced Second-Chance Algorithm





Additional-Reference-Bits Algorithm

- Referans bitlerini **düzenli zaman aralıklarıyla** kaydederek kullanım sırası hakkında ek bilgiler elde edebiliriz.
- Bellekte bir tablodaki her page için örneğin 8 bit referans biti kullanılsın, başlangıç değerleri sıfırdır. Bu 8 bitlik değer, son sekiz zaman dilimi için o page'in kullanımının geçmişini içerir.
- Her page'e referans yapıldığında o page'in 8bitlik referans bitinin en anlamlı biti 1 yapılır.
- Her zaman aralığının sonunda bitler bir sağa kaydırılır, en anlamlı bit 0 yapılır.
- Her bit bir zaman aralığına karşılık gelir, en soldaki bit en yakın zaman aralığını ve en sağdaki bit en önceki zaman aralığını gösterir.





Additional-Reference-Bits Algorithm

- Referans bitleri en düşük değer olan page LRU page'dir; yani boş frame kalmadığında belleğe getirilecek page, LRU page'in kullandığı frame'e geçecektir.
 - Bellekten çıkarılan page'in tüm bitleri sıfırlanır.
- Page'lerin benzersiz olmasının garantisi yoktur. En küçük aynı değerdeki tüm page'ler swap out yapılabilir veya aralarından seçim yapmak için FIFO yöntemini kullanılabilir.





Additional-Reference-Bits Algorithm

Reference String: 3,2,3, 8,0,3, 3,0,2, 6,3,4,7

Total Free Frames Allocated: 5

	F#	RB
0		0000
1		0000
2		0000
3		0000
4		0000
5		0000
6		0000
7		0000
8		0000

- Additional-Reference-Bits Algoritmasını, Reference Bit tablosundaki ve prosese ayrılan frame alanındaki güncellemeleri göstererek adım adım uygulayın.
- Reference string'teki üçlü gösterimler bir zaman aralığını temsil etmektedir, örneğin 1. zaman aralığında proses sırasıyla 3,2,3 page'lerine ihtiyaç duymaktadır.
- Her zaman aralığının sonunda ilgili page'in bit değerlerinde algoritmaya göre kaydırma işlemi yapılacaktır.

1 time interval
= 100 ms

0	
1	
2	
3	
4	



Additional-Reference-Bits Algorithm

Reference String: **3,2,3** 8,0,3, 3,0,2, 6,3,4,7

Total Free Frames Allocated: 5

F#	RB
0	0000
1	0000
2	1 0000 →
3	0 1 0000 →
4	0000
5	0000
6	0000
7	0000
8	0000

Shift
→
Right

	RB
0	0000
1	0000
2	0100
3	0100
4	0000
5	0000
6	0000
7	0000
8	0000

0	3
1	2
2	
3	
4	

1 time interval
= 100 ms





Additional-Reference-Bits Algorithm

Reference String: **3,2,3** **8,0,3** 3,0,2, 6,3,4,7

Total Free Frames Allocated: 5

F#	RB
0	0000
1	0000
2	1 0000 →
3	0 0000 →
4	0000
5	0000
6	0000
7	0000
8	0000

Shift
→
Right

F#	RB
0	3 0000
1	0000
2	1 0100
3	0 0100
4	0000
5	0000
6	0000
7	0000
8	2 0000

Right
Shift

F#	RB
0	3 0100
1	0000
2	1 0010
3	0 0110
4	0000
5	0000
6	0000
7	0000
8	2 0100

0	3
1	2
2	8
3	0
4	

1 time interval
= 100 ms





Additional-Reference-Bits Algorithm

Reference String: 3, 2, 3, 8, 0, 3, 3, 0, 2, 6, 3, 4, 7

Total Free Frames Allocated: 5

Handwritten notes: $1010 \rightarrow$ and a box containing 0101 .

F#	RB
0	0000
1	0000
2	1 0000
3	0 0000
4	0000
5	0000
6	0000
7	0000
8	0000

Shift Right

	RB
0	3 1 0000
1	0000
2	1 0100
3	0 100
4	0000
5	0000
6	0000
7	0000
8	2 1 0000

Right Shift

0	3 1 0100
1	0000
2	1 0010
3	0 0110
4	0000
5	0000
6	0000
7	0000
8	2 0100

Right Shift

0	0110
1	0000
2	0101
3	0111
4	0000
5	0000
6	0000
7	0000
8	0010

1 time interval
= 100 ms

0	3
1	2
2	8
3	0
4	

LRU

Order, LRU



Additional-Reference-Bits Algorithm

Reference String:

3, 2, 3

8, 0, 3

3, 0, 2

6, 3, 4, 7

Total Free Frames Allocated: 5

1010 →
0101

F#	RB
0	0000
1	0000
2	1 0000 →
3	0 0000
4	0000
5	0000
6	0000
7	0000
8	0000

Shift
→
Right

	RB
0	3 1 0000
1	0000
2	1 0100 4
3	0 0100
4	0000
5	0000
6	0000
7	0000
8	2 0000

Right
Shift

0	3 1 0100
1	0000
2	1 0010
3	0 0110
4	0000
5	0000
6	0000
7	0000
8	2 0100

Right
Shift

0	3 0110 6
1	0000
2	1 0101 5
3	0 0111 7
4	0000
5	0000
6	4 0000 8
7	0000
8	2 0010 2

0	3
1	2
2	8 → LRU
3	0
4	6

1 time interval
= 100 ms

{ Order,
LRU }



Additional-Reference-Bits Algorithm

Reference String: 3, 2, 3

Total Free Frames Allocated: 5

8, 0, 3

3, 0, 2

6, 3, 4, 7

1010 →
0101

F#	RB
0	0000
1	0000
2	<u>1</u> 0000 →
3	<u>0</u> 0000 →
4	0000
5	0000
6	0000
7	0000
8	0000

Shift
Right

	RB
0	<u>3</u> 0000
1	0000
2	<u>1</u> 0100
3	<u>0</u> 0100
4	0000
5	0000
6	0000
7	0000
8	<u>2</u> 0000

Right
Shift

0	<u>3</u> 0100
1	0000
2	<u>1</u> 0010
3	<u>0</u> 0110
4	0000
5	0000
6	0000
7	0000
8	<u>2</u> 0100

Right
Shift

0	<u>3</u> 0110
1	0000
2	<u>1</u> 0101
3	<u>0</u> 0111
4	<u>2</u> 0000
5	0000
6	<u>4</u> 0000
7	0000
8	<u>X</u> 0010

↓
0000

1 time interval
= 100 ms

0	<u>3</u>
1	<u>2</u>
2	<u>X</u> 4 → LRU
3	<u>0</u>
4	<u>6</u>

{ Order,
LRU }



Additional-Reference-Bits Algorithm

Reference String: 3, 2, 3, 8, 0, 3, 3, 0, 2, 6, 3, 4, 7

Total Free Frames Allocated: 5

1010 →
0101

101

F#	RB
0	0000
1	0000
2	1 0000 →
3	0 0000
4	0000
5	0000
6	0000
7	0000
8	0000

Shift Right

F#	RB
0	3 0000
1	0000
2	1 0100
3	0 0100
4	0000
5	0000
6	0000
7	0000
8	2 0000

Right Shift

F#	RB
0	3 0100
1	0000
2	1 0010
3	0 0110
4	0000
5	0000
6	0000
7	0000
8	2 0100

Right Shift

F#	RB
0	3 0110
1	0000
2	0x 0101
3	0 0111
4	2 0000
5	0000
6	4 0000
7	1 0000
8	x 0010

0000

0	3
1	x 7
2	x 4 → LRU
3	0
4	6

1 time interval
= 100 ms

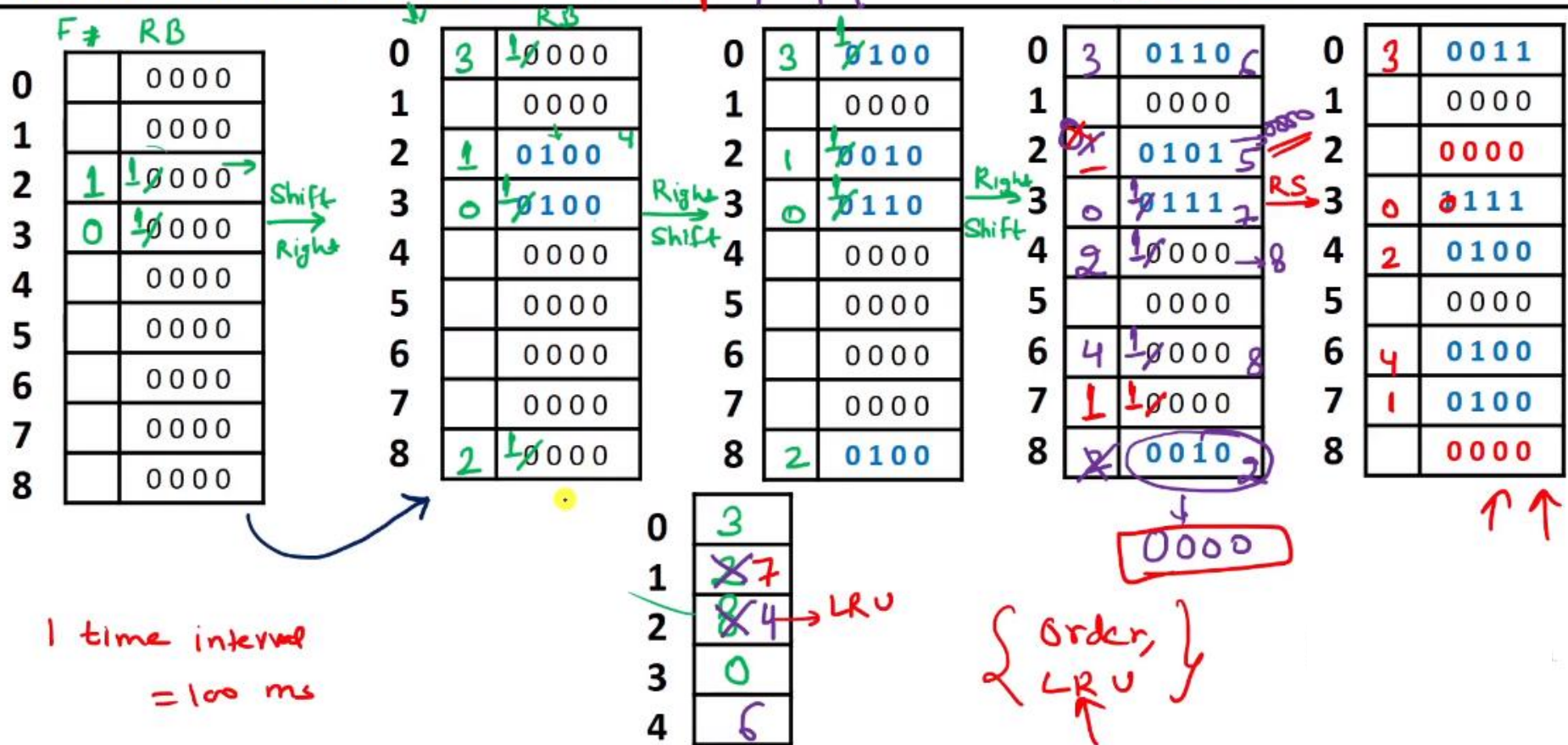
{ Order,
LRU }



Additional-Reference-Bits Algorithm

Reference String: 3,2,3 8,0,3 8,0,2 6,3,4,7
Total Free Frames Allocated: 5

1010 →
0101



1 time interval
= 100 ms

